PH 322 – Post Mid-Sem Lecture

Class Notes: Basics of Public-Key Cryptography and Shor's Algorithm

Dr. Shashank Gupta

March 23, 2026

Reference Texts

- P. Kaye, R. Laflamme, and M. Mosca, *An Introduction to Quantum Computing*, Oxford University Press.
- M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press.

Contents

1 Motivation: Why This Topic Matters

Public-key cryptography is one of the most remarkable ideas in modern information security. It allows two strangers to communicate securely over an insecure channel without first sharing a secret key. This is the mathematical foundation behind secure web browsing, digital signatures, encrypted email, online banking, software authentication, and much of modern public digital infrastructure.

For decades, the security of the most widely deployed public-key systems has relied on the apparent hardness of certain classical number-theoretic problems, especially:

- integer factorization, and
- discrete logarithms.

Shor's algorithm showed that a quantum computer can solve both of these problems efficiently. This discovery was historically revolutionary because it transformed quantum computing from a beautiful theoretical idea into a technology with immediate cryptographic consequences.

Central Message of This Lecture

The route from quantum mechanics to cryptographic impact runs through the following conceptual chain:

phase kickback $\longrightarrow$ phase estimation $\longrightarrow$ QFT $\longrightarrow$ order finding $\longrightarrow$ factoring/DL.

In earlier classes, we have already studied phase kickback, quantum phase estimation (QPE), and the quantum Fourier transform (QFT). The aim of the present lecture is to connect these ingredients into one coherent story and show how they culminate in Shor's algorithm.

2 Basics of Public-Key Cryptography

2.1 From Symmetric Keys to Public Keys

In **symmetric-key cryptography**, Alice and Bob share the same secret key for encryption and decryption. This raises a practical difficulty:

- How do Alice and Bob securely share a secret key before communication begins?

Public-key cryptography solves this key-distribution problem in a striking way. Each user possesses:

- a **public key**, which may be openly published, and
- a **private key**, which must remain secret.

The public key is used for encryption or verification, while the private key is used for decryption or signing.

2.2 Trapdoor One-Way Functions

The mathematical idea behind public-key cryptography is a **trapdoor one-way function**:

- easy to compute in the forward direction,
- believed hard to invert in general,
- but easy to invert if one possesses extra secret information called the *trapdoor*.

Examples of hard classical problems that support public-key systems include:

- factoring a large composite integer N,
- discrete logarithm in finite groups,
- elliptic-curve discrete logarithm.

2.3 Two Core Tasks

Public-key systems are generally used for two distinct purposes:

1. **Encryption:** Anyone can encrypt using the public key, but only the private-key holder can decrypt.
2. **Digital Signatures:** Only the private-key holder can sign, but everyone can verify using the public key.

Thus, public-key cryptography provides not only confidentiality but also authentication, integrity, and non-repudiation.

3 Mathematical Background for RSA

3.1 Modular Arithmetic

Much of public-key cryptography is built on arithmetic modulo an integer N.

For integers a and b, we write

$$a \equiv b \pmod{N}$$

if N divides $a - b$.

We work with the set

$$\mathbb{Z}_N = \{0, 1, 2, \ldots, N-1\},$$

with addition and multiplication performed modulo N.

3.2 Coprime Integers and Euler's Totient Function

Two integers a and N are called **coprime** if

$$\gcd(a, N) = 1.$$

The number of integers in $\{1, 2, \ldots, N-1\}$ that are coprime to N is denoted by Euler's totient:

$$\varphi(N).$$

If $N = pq$ where p and q are distinct primes, then

$$\varphi(N) = (p-1)(q-1).$$

3.3 Modular Inverses

If $\gcd(a, N) = 1$, then there exists an integer a^{-1} such that

$$aa^{-1} \equiv 1 \pmod{N}.$$

Such inverses are computed efficiently using the extended Euclidean algorithm. This simple classical fact is essential in RSA key generation.

3.4 Euler's Theorem

If $\gcd(a, N) = 1$, then

$$a^{\varphi(N)} \equiv 1 \pmod{N}.$$

This theorem underlies the correctness of RSA decryption.

4 RSA: The Standard Example

4.1 Key Generation

RSA is based on the difficulty of factoring a large composite number

$$N = pq,$$

where p and q are large primes.

The RSA setup proceeds as follows:

1. Choose two large primes p and q.
2. Compute
$$N = pq.$$
3. Compute
$$\varphi(N) = (p-1)(q-1).$$
4. Choose an integer e such that
$$\gcd(e, \varphi(N)) = 1.$$

5. Compute d such that
$$ed \equiv 1 \pmod{\varphi(N)}.$$

Then:

- **Public key:** (N, e)
- **Private key:** d

4.2 Encryption and Decryption

To encrypt a message $m \in \mathbb{Z}_N$, compute

$$c \equiv m^e \pmod{N}.$$

To decrypt, compute
$$m \equiv c^d \pmod{N}.$$

Indeed,
$$c^d \equiv (m^e)^d = m^{ed} \pmod{N}.$$

Since $ed \equiv 1 \pmod{\varphi(N)}$, one may write

$$ed = 1 + k\varphi(N)$$

for some integer k, so

$$m^{ed} = m^{1+k\varphi(N)} = m\left(m^{\varphi(N)}\right)^k \equiv m \pmod{N},$$

for messages coprime to N, and the full RSA construction can be shown to work more generally with standard padding conventions.

4.3 Why Factoring Matters

Observe the asymmetry:

- multiplying two primes p and q is easy,
- recovering p and q from $N = pq$ is believed hard classically.

But if an adversary can factor N, then they can compute

$$\varphi(N) = (p-1)(q-1),$$

and hence recover the private key d from the public information (N, e). Thus the security of RSA is intimately tied to the hardness of factoring.

5 Beyond RSA: Discrete Logarithms

5.1 The Discrete Logarithm Problem

A second major family of public-key cryptosystems is based on the **discrete logarithm problem**.

Given a group element a of order r, and another element

$$b = a^t,$$

the problem is to determine the integer t. We call t the discrete logarithm of b to the base a.

This problem arises in:

- Diffie–Hellman key exchange,
- ElGamal encryption,
- DSA-type signatures,
- elliptic-curve cryptography (ECC), where the group law is written additively.

5.2 Cryptographic Importance

Shor's quantum algorithm does not only threaten RSA. It also efficiently solves discrete logarithms in groups such as:

- multiplicative groups of finite fields,
- elliptic curve groups.

This is why the impact of quantum computing on current public-key cryptography is broad rather than narrow.

6 What We Already Know: The Three Quantum Ingredients

6.1 Phase Kickback

When a controlled unitary acts on an eigenstate of the target, the eigenvalue appears as a relative phase on the control system. Symbolically, if

$$U\left|u\right\rangle = e^{2\pi i\phi}\left|u\right\rangle,$$

then a controlled-U with control in superposition turns the phase $e^{2\pi i\phi}$ into measurable interference information.

This mechanism is the physical heart of quantum phase estimation.

6.2 Quantum Phase Estimation

Quantum phase estimation takes as input:

- a unitary U,
- an eigenstate $|u\rangle$,
- the promise that

$$U|u\rangle = e^{2\pi i\phi}|u\rangle .$$

The algorithm estimates ϕ by:

1. preparing a superposition in the control register,
2. applying controlled powers U^{2^j},
3. converting accumulated phase information into computational-basis information using the inverse QFT,
4. measuring.

6.3 Quantum Fourier Transform

The QFT is the transformation

$$\mathrm{QFT}_{2^n}|x\rangle = \frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} e^{2\pi i xy/2^n}|y\rangle .$$

Conceptually, the QFT is what turns hidden periodicity into sharp peaks in measurement statistics. That sentence is the bridge between abstract quantum algorithm design and number-theoretic problems such as factoring.

The Key Connection

If a computational problem can be reformulated as finding a hidden period, then QPE and QFT often provide a way to extract that period efficiently.

7 From Factoring to Order Finding

7.1 Definition of Order

Let a be an integer coprime to N. The **order** of a modulo N is the smallest positive integer r such that

$$a^r \equiv 1 \pmod{N}.$$

This r is a period of the function

$$f(x) = a^x \bmod N.$$

7.2 Why Order Finding Helps Factor N

This is the crucial number-theoretic reduction behind Shor's algorithm.

Suppose:

- N is odd and composite,
- a is chosen such that $\gcd(a, N) = 1$,
- r is the order of $a \pmod N$,
- r is even,
- $a^{r/2} \not\equiv -1 \pmod N$.

Then

$$a^r \equiv 1 \pmod N \quad \Longrightarrow \quad \left(a^{r/2} - 1\right)\left(a^{r/2} + 1\right) \equiv 0 \pmod N.$$

Hence N shares a nontrivial factor with at least one of

$$a^{r/2} - 1 \qquad \text{or} \qquad a^{r/2} + 1.$$

So we compute

$$\gcd\left(a^{r/2} - 1, N\right), \qquad \gcd\left(a^{r/2} + 1, N\right),$$

and obtain nontrivial factors with good probability.

7.3 A Small Example

Take $N = 15$ and choose $a = 2$. Since $\gcd(2, 15) = 1$, we may look for the order of 2 modulo 15.

We compute:

$$2^1 \equiv 2 \pmod{15}, \qquad 2^2 \equiv 4 \pmod{15}, \qquad 2^3 \equiv 8 \pmod{15}, \qquad 2^4 \equiv 16 \equiv 1 \pmod{15}.$$

So the order is

$$r = 4.$$

Now r is even, and

$$2^{r/2} = 2^2 = 4 \not\equiv -1 \pmod{15}.$$

Therefore,

$$\gcd(4 - 1, 15) = \gcd(3, 15) = 3,$$

and

$$\gcd(4 + 1, 15) = \gcd(5, 15) = 5.$$

Thus we recover the factorization

$$15 = 3 \times 5.$$

This example is tiny, but it illustrates the entire logic of Shor's factoring algorithm.

8 The Quantum Part: Order Finding as Eigenvalue Estimation

8.1 Constructing the Relevant Unitary

Define a unitary operator U_a by its action on basis states:

$$U_a |s\rangle = |as \bmod N\rangle ,$$

for s in the relevant computational basis and extended reversibly to a full unitary on the register.

Since a is coprime to N, multiplication by $a \bmod N$ is invertible, so U_a is indeed a valid permutation unitary.

8.2 Why This Helps

If r is the order of $a \pmod N$, then

$$U_a^r = I$$

on the cyclic subspace generated by modular multiplication. Therefore, the eigenvalues of U_a are r-th roots of unity:

$$e^{2\pi ik/r}, \qquad k = 0, 1, \ldots, r-1.$$

This is the moment where number theory meets quantum mechanics.

Big Idea

To find the order r, it is enough to estimate phases of the form

$$\phi = \frac{k}{r}.$$

Once we obtain sufficiently accurate estimates of random fractions k/r, classical post-processing recovers r.

8.3 Eigenstates of U_a

For each $k \in \{0, 1, \ldots, r-1\}$, define

$$|u_k\rangle = \frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} e^{-2\pi iks/r} |a^s \bmod N\rangle .$$

One can verify that

$$U_a |u_k\rangle = e^{2\pi ik/r} |u_k\rangle .$$

Thus the phase associated with $|u_k\rangle$ is precisely k/r.

8.4 Where Phase Kickback Enters

Now recall the earlier story from class.

If we apply controlled powers $U_a^{2^j}$ to an eigenstate $|u_k\rangle$, then

$$U_a^{2^j} |u_k\rangle = e^{2\pi i 2^j k/r} |u_k\rangle .$$

By phase kickback, these eigenvalue phases accumulate in the control register. The inverse QFT then decodes that phase into an estimate of the binary expansion of

$$\phi = \frac{k}{r}.$$

So Shor's algorithm is not an isolated trick. It is a highly structured application of the exact machinery you have already studied:

$$\text{controlled powers} + \text{phase kickback} + \text{QPE} + \text{QFT}^{-1}.$$

9 The Order-Finding Circuit

9.1 Algorithmic Structure

The quantum order-finding routine proceeds as follows.

1. Choose an integer a with $1 < a < N$ and $\gcd(a, N) = 1$.
2. Use one register as a control register and prepare it in a uniform superposition.
3. Prepare the second register in a simple computational basis state, usually $|1\rangle$.
4. Apply the modular exponentiation unitary:
$$|x\rangle |1\rangle \mapsto |x\rangle |a^x \bmod N\rangle .$$
5. Apply the inverse QFT to the first register.
6. Measure the first register.

This is often presented as Shor's algorithm in its original form. Conceptually, it is equivalent to an eigenvalue-estimation view of the same circuit.

9.2 Why the Measurement Contains Period Information

The state

$$\frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} |x\rangle |a^x \bmod N\rangle$$

contains a periodic structure because the second register repeats with period r. After the inverse QFT on the first register, the amplitude distribution develops peaks near integer multiples of Q/r, where Q is the size of the control register space.

Thus measurement outcomes x satisfy, with high probability,

$$\frac{x}{Q} \approx \frac{k}{r}$$

for some integer k.

9.3 Continued Fractions

Once the measured value x is obtained, we classically process the rational approximation

$$\frac{x}{Q}$$

using the continued fractions algorithm. This allows us to recover a candidate denominator r, provided the estimate is sufficiently accurate.

So the full architecture is:

$$\text{quantum subroutine gives } \frac{k}{r} \quad \longrightarrow \quad \text{classical continued fractions gives } r.$$

This is an elegant hybrid quantum-classical algorithm:

- the **hard periodicity extraction** is quantum,
- the **number-theoretic cleanup** is classical.

10 Shor's Factoring Algorithm: Complete Workflow

10.1 Full Procedure

We may now summarize Shor's factoring algorithm for an odd composite integer N.

1. Pick a random integer a such that $1 < a < N$.
2. Compute $\gcd(a, N)$.
   - If $\gcd(a, N) > 1$, then we have already found a nontrivial factor.
   - Otherwise continue.
3. Use the quantum order-finding routine to find the order r of a modulo N.
4. If r is odd, declare failure and try again with another a.
5. Compute $a^{r/2} \bmod N$.
6. If
   $$a^{r/2} \equiv -1 \pmod{N},$$
   declare failure and try again with another a.
7. Otherwise compute
   $$\gcd(a^{r/2} - 1, N) \qquad \text{and} \qquad \gcd(a^{r/2} + 1, N).$$
   These yield nontrivial factors of N with high probability.

10.2 Why the Algorithm is Efficient

The quantum part reduces factoring to order finding, and order finding is solved efficiently by QPE/QFT-based methods. The Mosca text emphasizes that the computational bottleneck is modular exponentiation, but it can still be implemented with a number of elementary gates polynomial in $\log N$. Thus the overall factoring algorithm is efficient in the input length, which is the number of bits of N.

This is the essential contrast with the best known classical factoring algorithms, which are not known to run in polynomial time.

11 Why This Breaks Public-Key Cryptography

11.1 RSA

If Shor's algorithm factors $N = pq$, then RSA private keys can be derived from public information:

1. factor N,
2. compute $\varphi(N) = (p-1)(q-1)$,
3. compute $d = e^{-1} (\text{mod } \varphi(N))$.

Thus, a sufficiently large fault-tolerant quantum computer would undermine RSA security.

11.2 Discrete-Log Systems

The same chapter in Kaye–Laflamme–Mosca also explains that Shor's framework extends to the discrete logarithm problem. Therefore, systems based on finite-field discrete logarithms and elliptic-curve discrete logarithms are also vulnerable in principle.

This means that a cryptographically relevant quantum computer would threaten:

- RSA,
- Diffie–Hellman,
- ElGamal,
- DSA,
- elliptic-curve cryptography.

11.3 A Historical Remark

This is one of the main reasons quantum computing received intense attention from governments, industry, and the cryptographic community. Shor's algorithm did not merely give a faster algorithm for a mathematical puzzle; it challenged the security assumptions behind real-world digital infrastructure.

12 How to Explain Shor's Algorithm Intuitively

Students often find the formal circuit clear but the conceptual story harder to hold together. The following picture is useful.

Intuition

Classically, the sequence

$$a, a^2, a^3, a^4, \ldots \pmod{N}$$

looks like a hard number-theoretic process. Quantum mechanically, we turn that sequence into a periodic wave. The QFT then acts like a frequency analyzer, revealing the hidden period.

So one may think of Shor's algorithm as:

1. build a periodic pattern in amplitudes,
2. use interference to reveal the frequency,
3. translate the frequency back into number-theoretic structure.

This is why the QFT is central. It is not just another gate decomposition; it is the mechanism that converts hidden periodicity into accessible measurement data.

13 Circuit-Level Connection to Earlier Topics

13.1 From Deutsch–Jozsa to Shor

In the Deutsch and Deutsch–Jozsa algorithms, the final Hadamard transform extracts information encoded in phases. The Mosca text explicitly uses this as motivation for phase estimation. The conceptual leap is that Shor's algorithm performs the same basic maneuver on a far richer phase structure.

13.2 From Phase Estimation to Order Finding

In QPE, the unknown quantity is an eigenphase ϕ. In Shor's algorithm, the relevant unitary is modular multiplication U_a, whose eigenphases are

$$\phi = \frac{k}{r}.$$

So:

- QPE estimates k/r,
- continued fractions recovers r,
- number theory turns r into factors of N.

This is the cleanest way to "connect the dots":

$$\boxed{\text{phase kickback} \Rightarrow \text{QPE} \Rightarrow \text{eigenphase } k/r \Rightarrow \text{order } r \Rightarrow \text{factors of } N}$$

14 Practical Notes for the Qiskit Demonstration

14.1 What Students Should Watch For

In a Qiskit demonstration of Shor's algorithm, students should pay attention to the following components:

- the control register prepared in superposition,
- the modular exponentiation circuit,
- the inverse QFT,
- the measured bit string in the phase register,
- the classical continued-fractions post-processing,
- the final extraction of factors via GCD.

14.2 Why Small Demonstrations Are Still Valuable

In classroom demonstrations, one usually factors small numbers such as 15 or 21, because full-scale fault-tolerant Shor computation is far beyond current noisy hardware. But the pedagogical value is high:

- students see the hybrid quantum-classical structure,
- the role of inverse QFT becomes concrete,
- the relation between measured phase and order becomes visible,
- the cryptographic significance is immediate.

14.3 A Good Classroom Narrative

A good way to narrate the live demo is:

1. "We are not directly factoring."
2. "We are using quantum mechanics to learn a period."
3. "That period is the order r."
4. "Number theory converts r into the factors."

That framing helps students see Shor's algorithm as a synthesis of physics, linear algebra, interference, and arithmetic.

15 Limitations and Modern Perspective

15.1 What Shor's Algorithm Does *Not* Mean

Shor's algorithm does *not* imply that quantum computers solve all hard problems efficiently. Its power is highly structured: it exploits algebraic periodicity. Problems without such structure, such as generic unstructured search, do not enjoy exponential speedup from the same mechanism.

15.2 Cryptographic Consequence

The correct lesson is therefore not "all cryptography is broken," but rather:

- many widely deployed *public-key* systems are vulnerable in principle,
- this is why **post-quantum cryptography** has become urgent,
- symmetric cryptography is affected much less dramatically, mainly through quadratic-speedup results such as Grover's algorithm.

16 Summary

- Public-key cryptography relies on computational asymmetry: easy forward computation, hard inversion.
- RSA is based on integer factorization, while many other systems are based on discrete logarithms.
- Shor's algorithm reduces factoring to order finding.
- Order finding is solved quantum mechanically by estimating eigenphases of modular multiplication unitaries.
- Phase kickback, quantum phase estimation, and the inverse QFT are the core quantum ingredients.
- Classical post-processing via continued fractions and GCD completes the factorization.

The deepest conceptual point is this: *Shor's algorithm is a phase-estimation algorithm in disguise.* Once that is understood, the entire subject becomes far more coherent.

Exercises

Exercise 1: RSA Warm-up
Let $p = 11$ and $q = 17$.

(a) Compute $N = pq$ and $\varphi(N)$.

(b) Choose $e = 7$. Verify that $\gcd(e, \varphi(N)) = 1$.

(c) Find the private exponent d satisfying $ed \equiv 1 \pmod{\varphi(N)}$.

(d) Encrypt the message $m = 8$.

(e) Decrypt the ciphertext and verify that the original message is recovered.

Exercise 2: Orders Modulo N
For each pair (a, N), compute the order of a modulo N:

(a) $a = 2$, $N = 15$

(b) $a = 2$, $N = 21$

(c) $a = 7$, $N = 15$

(d) $a = 10$, $N = 21$

Exercise 3: From Order to Factors

Suppose the order of a modulo N is r, with r even. Show that if $a^{r/2} \not\equiv -1 \pmod{N}$, then

$$\gcd(a^{r/2} - 1, N) \quad \text{and} \quad \gcd(a^{r/2} + 1, N)$$

yield nontrivial factors of N. Apply this explicitly for:

(a) $N = 21$, $a = 2$

(b) $N = 33$, $a = 10$

Exercise 4: Phase Kickback Review

Suppose $U|u\rangle = e^{2\pi i\phi}|u\rangle$. Show that if the control qubit is initialized in

$$\frac{|0\rangle + |1\rangle}{\sqrt{2}},$$

then after a controlled-U operation the joint state becomes

$$\frac{1}{\sqrt{2}}\left(|0\rangle|u\rangle + |1\rangle e^{2\pi i\phi}|u\rangle\right).$$

Explain in words why this is called *phase kickback*.

Exercise 5: QPE Interpretation

Suppose the eigenphase is

$$\phi = \frac{3}{8}.$$

If ideal QPE is performed using three control qubits, what measurement result should be obtained? Explain how this connects to the binary expansion of ϕ.

Exercise 6: Modular Multiplication Unitary

Let $N = 15$ and $a = 2$. Write the action of the unitary

$$U_a|s\rangle = |2s \bmod 15\rangle$$

on the basis states $|1\rangle, |2\rangle, |4\rangle, |8\rangle$. Show explicitly that repeated application cycles through these states and identify the order r.

Exercise 7: Continued Fractions Step

Suppose a control register of size $Q = 256$ yields the measured value $x = 64$.

(a) Compute x/Q.

(b) Express it as a rational number in lowest terms.

(c) Interpret the denominator as a candidate order.

Repeat the same for $x = 85$ and discuss why rational approximation is needed in realistic cases.

Exercise 8: Why RSA Fails if Factoring is Easy
Assume an adversary knows the RSA public key (N, e), and can factor N. Describe the full chain of classical computations needed to recover the private key d. Write each step clearly.

Exercise 9: Discrete Logarithm Cryptography
Research briefly the role of the discrete logarithm problem in one of the following:

- Diffie–Hellman key exchange,
- ElGamal encryption,
- elliptic-curve cryptography.

Then explain in one page why Shor's discrete-log algorithm is cryptographically significant.

Exercise 10: Qiskit Reflection
After the classroom demonstration of Shor's algorithm in Qiskit, identify:

(a) which part of the circuit corresponds to modular exponentiation,

(b) which part corresponds to inverse QFT,

(c) what classical post-processing is done after measurement,

(d) why the measured bit string does not directly equal a factor.

References

References

[1] P. Kaye, R. Laflamme, and M. Mosca, *An Introduction to Quantum Computing*, Oxford University Press.

[2] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press.

[3] R. L. Rivest, A. Shamir, and L. Adleman, "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems," *Communications of the ACM*, 21(2), 1978.

[4] P. W. Shor, "Algorithms for Quantum Computation: Discrete Logarithms and Factoring," *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*, 1994.